CSE142 Object Oriented Programming Techniques (Spring'25)

Final Examination

Max Marks: 100

Time Allowed: 3 hours

Instructions:

- Answer the questions in the spaces provided on the question sheets
- Make sure to write your name and ERP on each page
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam
- Make sure you have all 19 pages (printed on 10 two sided sheets) of the exam before you begin

Name:_____ **ERP:** _____

Solution

Parts:	Part I (Tracing)	Part II (Short Questions)	Part III (Implementation)	Total
Marks:	28	27	45	100
Score:				

Part I (Tracing)

Q1: [14 marks] What will be the output of the following programs? If the program has an error, write `Error` and explain the error. Each part carries 2 marks.

Assume that the relevant standard headers and namespaces are already available.

(a) [2 marks]

```
class Item {
    int value;
public:
    Item(int v) : value(v) {}
    bool operator==(const Item& other) {
        return this->value == other.value;
    }
};
int main() {
    Item a(5), b(5);
    cout << (a == b ? "Not Equal" : "Equal");
}
```

```
Not Equal
```

(b) [2 marks]

```
class Number {
    int n;
public:
    Number(int val = 0) : n(val) {}
    operator int() const { return n; }

    int Sum() const {
        if (n < 0)
            throw std::invalid_argument("Negative number not allowed");
        if (n == 0) return 0;
        return n + Number(n - 1).Sum();
    }
};

int main() {
```

```
        try {
            Number x = 5;
            cout << x.Sum() << endl;
            Number y = -3;
            cout << y.Sum() << endl;
        } catch (const exception& e) {
            cout << "Exception: " << e.what() << endl;
        }
    }
```

```
15
Exception: Negative number not allowed
```

(c) [2 marks]

```
class Sorter {
  public:
      static void sort(int arr[], int size) {
          for (int i = 1; i < size; ++i) {
              int key = arr[i], j = i - 1;
              while (j >= 0 && arr[j] < key)
                  arr[j + 1] = arr[j--];
              arr[j + 1] = key;
          }
      }
  };
  int main() {
      int data[] = {5, 3, 4, 1};
      Sorter::sort(data, 4);
      for (int i : data) cout << i << ' ';
  }
```

```
5 4 3 1
```

(d) [2 marks]

```
class Sequence {
      int* arr;
      int size;
public:
  Sequence(int s) : size(s) { arr = new int[size]; }
  ~Sequence() { delete[] arr; }
  void fill() { for(int i = 0; i < size; ++i) arr[i] = i*i; }
   void display() const { for(int i = 0; i < size; ++i) cout << arr[i] << ' '; }
};
int main() {
      Sequence sq(4);
      sq.fill();
      sq.display();
}
```

```
0 1 4 9
```

(e) [2 marks]

```
void mergeSort(int a[], int l, int r) {
      if (l >= r) return;
      int m = (l + r) / 2;
      mergeSort(a, l, m);
```

```
        mergeSort(a, m+1, r);
    }
    int main() {
        int a[] = {1,4,2,3};
        mergeSort(a, 0, 3);
        for (int i : a) cout << i << ' ';
    }
```

1 4 2 3 (unchanged – merge logic missing)

(f) [2 marks]

```
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; ++j)
        if (a[j] > pivot) swap(a[++i], a[j]);
    swap(a[i+1], a[high]);
    return i + 1;
}
void quickSort(int a[], int l, int r) {
    if (l < r) {
        int pi = partition(a, l, r);
        quickSort(a, l, pi - 1);
        quickSort(a, pi + 1, r);
    }
}
int main() {
    int a[] = {1,4,2,3};
    quickSort(a, 0, 3);
    for (int i : a) cout << i << ' ';
}
```

4 3 2 1

(g) [2 marks]

```
void solve(int n, string out, int open, int close) {
    if (out.length() == 2*n) {
        cout << out << endl;
        return;
    }
    if (open < n) solve(n, out + "(", open+1, close);
    if (close < open) solve(n, out + ")", open, close+1);
}
int main() {
    solve(2, "", 0, 0);
}
```

(())
()()

Q2: [14 marks] Given the input array: $A = [38, 27, 43, 3, 30, 82, 10]$
Simulate the Shell Sort algorithm using the gap sequence:

$$\texttt{gap} = 3 \rightarrow 1 \rightarrow 0$$

Complete the following table by performing Shell Sort. For each value of '**gap**', simulate the inner insertion sort logic by filling in the current values of '**i**', '**j**', '**temp**', and the array state after each significant step. Use '**swap**' to indicate when a value is moved backward, and '**stop**' when the condition fails and no further swaps are made. Each row in the table carries 1 mark.

Note: The actual implementation code is not provided intentionally. You are expected to deduce the specific values of 'i', 'j', 'temp', the condition being checked, and the resulting array state at each step based on your understanding of the Shell Sort algorithm.

Gap	i	j	temp	Condition Checked	Array State
3	3	0	3	A[0] (=38) > 3 → swap	[3, 27, 43, 38, 30, 82, 10]
3	4	1	30	A[1] (=27) < 30 → stop	[3, 27, 43, 38, 30, 82, 10]
3	5	2	82	A[2] (=43) < 82 → stop	[3, 27, 43, 38, 30, 82, 10]
3	6	3	10	A[3] (=38) > 10 → swap	[3, 27, 43, 10, 30, 82, 38]
3	6	0	10	A[0] (=3) < 10 → stop	[3, 27, 43, 10, 30, 82, 38]
Gap = 1					
1	1	0	27	A[0] (=3) < 27 → stop	[3, 27, 43, 10, 30, 82, 38]
1	2	1	43	A[1] (=27) < 43 → stop	[3, 27, 43, 10, 30, 82, 38]
1	3	2	10	A[2] (=43) > 10 → swap	[3, 27, 10, 43, 30, 82, 38]
1	3	1	10	A[1] (=27) > 10 → swap	[3, 10, 27, 43, 30, 82, 38]
1	3	0	10	A[0] (=3) < 10 → stop	[3, 10, 27, 43, 30, 82, 38]
1	4	3	30	A[3] (=43) > 30 → swap	[3, 10, 27, 30, 43, 82, 38]
1	4	2	30	A[2] (=27) < 30 → stop	[3, 10, 27, 30, 43, 82, 38]
1	5	4	82	A[4] (=43) < 82 → stop	[3, 10, 27, 30, 43, 82, 38]
1	6	5	38	A[5] (=82) > 38 → swap	[3, 10, 27, 30, 43, 38, 82]
1	6	4	38	A[4] (=43) > 38 → swap	[3, 10, 27, 30, 38, 43, 82]
1	6	3	38	A[3] (=30) < 38 → stop	[3, 10, 27, 30, 38, 43, 82]
Gap = 0 → Sorting complete					

Part II (Short Questions)

Q3: [5 marks] Match each concept (numbered 1–10) with the correct definition below. For each definition, write the number (1–10) of the matching concept in the box before it. Only write the number of the concept (not letters or text). Each concept carries 0.5 marks.

Note: Overwriting or cutting is not allowed. Make up your mind before writing the answer. Any scribbling will result in the award of zero marks.

Concepts

1. Generic Programming
2. Template Function
3. Template Class
4. Abstract Class
5. Pure Virtual Function
6. Inheritance
7. Multiple Inheritance
8. Run-Time Polymorphism
9. Interface Segregation Principle (ISP)
10. Compile-Time Polymorphism

Definitions

[7] A feature where a class can inherit from more than one base class

[4] A class that cannot be instantiated and may contain pure virtual methods

[9] Designing systems so that classes depend only on the methods they use

[3] A class template that generates a family of types for multiple data types

[10] The ability to resolve calls to different functions or operations during compilation, such as through overloading or templates

[8] The ability of different types of objects to respond to the same function call in a type-specific way, determined at runtime

[6] Inheriting members from a base class to promote modularity and reuse

[2] A function written using templates to work with different data types

[5] Defining behavior in a base class to be implemented in derived classes

[1] A style of programming that uses templates to work with any data type

Q4: [11 marks] **You are given a stack implementation using a singly linked list.** The current implementation of `push` and `pop` operates at the **rear**, which is inefficient due to traversal. You need to optimize these operations by modifying only their bodies. **Do NOT change the class definition or its data members.**

Provided Code (Inefficient Implementation):

```
class LLStack {
private:
    struct Node {
        string data;
        Node* next;
    };
    Node* first = nullptr;
    int N = 0;

public:
    void push(string item) {
```

```
        if (first == nullptr) {
            first = new Node{ item, nullptr };
            N++;
            return;
        }
        Node* current = first;
        while (current->next != nullptr) {
            current = current->next;
        }
        current->next = new Node{ item, nullptr };
        N++;
    }

    string pop() {
        if (first == nullptr) return "";
        if (first->next == nullptr) {
            string data = first->data;
            delete first;
            first = nullptr;
            N--;
            return data;
        }
        Node* current = first;
        while (current->next->next != nullptr) {
            current = current->next;
        }
        string data = current->next->data;
        delete current->next;
        current->next = nullptr;
        N--;
        return data;
    }
};
```

(a) [1 mark] What is the internal state of the linked list just before the `while` loop starts in the following `main` function?

```
int main() {
    LLStack s;
    s.push("4");
    s.push("3");
    s.push("2");
    s.push("1")?

    string output = "test";
    while (!output.empty()) {
        output = s.pop();
        cout << output;
    }
}
```

Show the data values in the order of nodes from `first` to `nullptr` in the inefficient version:

first → 4 → 3 → 2 → 1 → nullptr

(b) [1 mark] What will be the output of the program with the inefficient version?

1234

(c) [1 mark] What is the time complexity of `push` and `pop` in the inefficient version?

push: O(n)
pop: O(n)

(d) [5 marks] Rewrite only the bodies of `push` and `pop` functions to make them efficient (no traversal). **Do not modify data members or class structure.**

```
void push(string item) {
```

```
    Node* second = first;
    first = new Node{item, second};
    N++;
```

```
}
string pop() {
```

```
    Node* second = first->next;
    string d = first->data;
    delete first;
    first = second;
    N--;
    return d;
```

```
}
```

(e) [1 mark] Now, what will be the internal state of the linked list just before the `while` loop in the `main` starts?
Show the data values in the order of nodes from `first` to `nullptr` in the efficient version:

first → 1 → 2 → 3 → 4 → nullptr

(f) [1 mark] What will be the output of the program with the efficient version?

1234

(g) [1 mark] What is the time complexity of `push` and `pop` in the efficient version?

push: O(1)
pop: O(1)

Q5: [11 marks] **You are given a queue implementation using a resizable array.** The current implementation **does not reuse freed space** when items are dequeued. You must optimize only the `enqueue` and `dequeue` function bodies to make the queue circular and efficiently resizable. **Do NOT change the class definition or data members.**

Provided Code (Inefficient Implementation):

```
class RAQueue {
private:
   int sz;
   int cap;
   int front;
   int rear;
   string* arr = nullptr;

public:
   RAQueue() : sz{ 0 }, cap{ 2 }, front{ 0 }, rear{ 0 }, arr{ new string[cap] } {}

   void enqueue(string item) {
      if (rear == cap) resize(2 * cap);
      arr[rear++] = item;
      sz++;
   }

   string dequeue() {
      if (sz == 0) return "";
      string item = arr[front];
      arr[front] = "";
      front++;
      sz--;
      return item;
   }

private:
   void resize(int newcap) {
      string* newarr = new string[newcap];
      for (int i = 0; i < sz; ++i)
         newarr[i] = arr[front + i];
      delete[] arr;
      arr = newarr;
      front = 0;
      rear = sz;
      cap = newcap;
   }
};
```

(a) [2 marks] What is the internal state of the array just before the `while` loop starts in the following `main` function?

```
int main() {
   RAQueue q;
   q.enqueue("C");
   q.enqueue("D");
   q.enqueue("A");
   q.enqueue("B");
```

```
  q.enqueue(q.dequeue());
  q.enqueue(q.dequeue());

  string output = "start";
  while (!output.empty()) {
    output = q.dequeue();
    cout << output;
  }
}
```

Use [-] for cleared or unused entries. Also, state the values of `cap`, `front`, `rear`, and `sz` in the inefficient version:

```
Index:   [ 0 ]  [ 1 ]  [ 2 ]  [ 3 ] [ 4 ] [ 5 ] [ 6 ] [ 7 ]
Value:   [ - ]  [ A ]  [ B ]  [ C ] [ D ] [ - ] [ - ] [ - ]

 cap   = 8
 front = 1
 rear  = 5
 sz    = 4
```

(b) [1 mark] What will be the output of the program with the inefficient version?

ABCD

(c) [5 marks] Rewrite only the bodies of `enqueue` and `dequeue` to make the queue efficient using circular indexing and dynamic resizing. **Do not modify data members or class structure.**

```
void enqueue(string item) {
    if (sz == cap) resize(2 * cap);
    arr[rear] = item;
    rear = (rear + 1) % cap;
    sz++;
}
```

Name/ERP: _____

```
string dequeue() {
```

```
    if (sz == 0) return "";
    string item = arr[front];
    arr[front] = "";
    front = (front + 1) % cap;
    sz--;
    if (sz > 0 && sz == cap / 4) resize(cap / 2);
    return item;
```

}

(d) [2 marks] Now, what will be the internal state of the array just before the `while` loop in the `main` starts?

Use [-] for cleared or unused entries. Also, state the values of `cap`, `front`, `rear`, and `sz` in the efficient version:

```
Index:   [ 0 ]  [ 1 ]  [ 2 ]  [ 3 ] [ 4 ] [ 5 ] [ 6 ] [ 7 ]
Value:   [ C ]  [ D ]  [ A ]  [ B ] [ - ] [ - ] [ - ] [ - ]

 cap   = 4
 front = 2
 rear  = 2
 sz    = 4
```

(e) [1 mark] What will be the output of the program with the efficient version?

ABCD

Part III (Implementation)

Q6: [10 marks] You are given a grayscale image represented as a 2D matrix of integers, where each value represents a pixel's brightness and lies between 0 (black) and 255 (white).

Requirements

You are required to apply pixel-wise transformations to the image. These transformations must be implemented using:

- A regular function
- A functor
- A lambda expression

Each transformation will be passed into a generic function (`transformImage, whose` header is given below), which applies it to every pixel in the image and returns a new image matrix.

Each of the following three transformations must be implemented using **one of the above callable types (each used once)**:

1. Brightening (1 mark)
    - Increase the brightness of each pixel by +50.
    - Ensure the value does not exceed 255
    - Implement this transformation using a **regular function**
2. Inverting (1 mark)
    - Replace each pixel value with 255 - pixel (i.e., black becomes white, and white becomes black)
    - Implement this transformation using a **functor**
3. Contrast Enhancement (1 mark)
    - Multiply each pixel by itself
    - Clamp the result to a maximum of 255
    - Implement this transformation using a **lambda expression**

Function Requirements

Implement the following template function: (4 marks)

```
template <typename T>
std::vector<std::vector<int>> transformImage(const std::vector<std::vector<int>>& image, T transform);
```

This function should:

- Accept a 2D image matrix
- Apply `transform(pixel)` to each pixel
- Return a new matrix with the transformed values
- Leave the original image unchanged

Additional Requirements

- Write a `main()` function to demonstrate all three transformations on a sample 2D image matrix (2 marks)
- Write a helper function `printImage` to print the image matrix, and call it in the `main()` function to print all three transformed images (1 mark)

```
#include <iostream>
#include <vector>

template <typename T>
std::vector<std::vector<int>> transformImage(const std::vector<std::vector<int>>& image, T transform) {
    std::vector<std::vector<int>> result = image;

    for (int i = 0; i < result.size(); ++i) {
        for (int j = 0; j < result[i].size(); ++j) {
            result[i][j] = transform(result[i][j]);
        }
    }

    return result;
}

int brighten(int pixel) {
    return std::min(pixel + 50, 255);
}
```

```
class Invert {
public:
    int operator()(int pixel) const {

        return 255 - pixel;
    }
};
void printImage(const std::vector<std::vector<int>>& image) {
    for (const auto& row : image) {
        for (int pixel : row) {
            std::cout << pixel << " ";
        }
        std::cout << "\n";
    }
    std::cout << std::endl;
}
int main() {
    std::vector<std::vector<int>> image = {
        { 50, 100, 150 },
        { 200, 220, 240 },
        { 10, 30, 60 }
    };

    auto brightened = transformImage(image, brighten);
    printImage(brightened);

    auto inverted = transformImage(image, Invert());
    printImage(inverted);

    auto contrasted = transformImage(image, [](int pixel) {
        return std::min(pixel * pixel, 255);
    });
    printImage(contrasted);

    return 0;
}
```

Name/ERP: _____

Q7: [15 marks] Imagine that you are building an **auto-grading system** that manages student submissions. Each submission includes a student's name and score.
To handle these submissions, the system uses a class hierarchy as follows:

1. **SubmissionManager –** an abstract base class that defines the general interface
2. **SubmissionList –** a concrete class that implements the interface using a **dynamic array** to store submissions in the order they arrive

You are given the code for the above classes. Your task is to implement a new class called **SortedLeaderboard**, which stores submissions **in ascending order of score**.

Provided Code:

```
#include <iostream>
#include <string>
using namespace std;

// Class to hold name and score
class Submission {
public:
    string name;
    int score;

    Submission(string n = "", int s = -1) {
        name = n;
        score = s;
    }
};

// Abstract Base Class
class SubmissionManager {
public:
    virtual void add(string name, int score) = 0;
    virtual Submission remove() = 0;
    virtual Submission peek() const = 0;
    virtual bool isEmpty() const = 0;
    virtual ~SubmissionManager() {}
};

// Concrete Class: SubmissionList (unsorted, dynamic array)
class SubmissionList : public SubmissionManager {
protected:
    Submission* submissions;
    int size;
    int capacity;

    void resize(int newCapacity) {
        Submission* newArray = new Submission[newCapacity];
        for (int i = 0; i < size; ++i) {
            newArray[i] = submissions[i];
        }
        delete[] submissions;
        submissions = newArray;
        capacity = newCapacity;
    }

public:
    SubmissionList() {
```

```
        capacity = 10;
        size = 0;
        submissions = new Submission[capacity];
    }

    virtual ~SubmissionList() {
        delete[] submissions;
    }

    void add(string name, int score) override {
        if (size == capacity) {
            resize(capacity * 2); // Grow when full
        }
        submissions[size++] = Submission(name, score);
    }

    Submission remove() override {
        if (isEmpty()) return Submission("", -1);
        Submission first = submissions[0];
        for (int i = 1; i < size; ++i) {
            submissions[i - 1] = submissions[i];
        }
        size--;
        if (size < capacity / 4 && capacity > 10) {
            resize(capacity / 2); // Shrink when mostly empty
        }
        return first;
    }

    Submission peek() const override {
        if (isEmpty()) return Submission("", -1);
        return submissions[0];
    }

    bool isEmpty() const override {
        return size == 0;
    }
};
```

Implement a new class called **SortedLeaderboard** that:

- Inherits from SubmissionList (2 marks)
- Overrides the **add()** function to insert submissions in **ascending order of score (8 marks)**
- Implements a function **void printFormatted() const** that prints the current state of the leaderboard in the format shown below (2 marks)

Note the following points:

- You **do not** need to override or modify remove(), peek(), or isEmpty() — simply inherit their behavior as-is.
- The provided resize(int newCapacity) function is already used inside add() and remove() in the base class. You may use it if needed in your override of add() when the array is full.

Additional Requirement Write a **main()** function that: (3 marks)

- Creates a SortedLeaderboard object
- Adds the following submissions in this order:

```
["Ali", 72]
["Zara", 88]
["Bilal", 64]
["Sana", 91]
["Daniyal", 81]
```

- After each addition, calls **printFormatted()** to show the current state of the leaderboard:

Example Output

```
{}
{["Ali", 72]}
{["Ali", 72], ["Zara", 88]}
{["Bilal", 64], ["Ali", 72], ["Zara", 88]}
{["Bilal", 64], ["Ali", 72], ["Zara", 88], ["Sana", 91]}
{["Bilal", 64], ["Ali", 72], ["Daniyal", 81], ["Zara", 88], ["Sana", 91]}
```

```
class SortedLeaderboard : public SubmissionList {
public:
    void add(string name, int score) override {
        if (size == capacity) {
            resize(capacity * 2);
        }

        Submission newSubmission(name, score);
        int insertPos = 0;

        // Find position where the new score fits
        while (insertPos < size && submissions[insertPos].score <= score) {
            insertPos++;
        }

        // Shift elements to make space
        for (int i = size; i > insertPos; --i) {
            submissions[i] = submissions[i - 1];
        }

        submissions[insertPos] = newSubmission;
        size++;
    }

    void printFormatted() const {
        cout << "{";
        for (int i = 0; i < size; ++i) {
            cout << "[\"" << submissions[i].name << "\", " <<
submissions[i].score << "]";
            if (i != size - 1)
                cout << ", ";
        }
        cout << "}" << endl;
    }
};
```

```
int main() {
    SortedLeaderboard leaderboard;

    leaderboard.printFormatted(); // {}

    leaderboard.add("Ali", 72);
    leaderboard.printFormatted();

    leaderboard.add("Zara", 88);
    leaderboard.printFormatted();

    leaderboard.add("Bilal", 64);
    leaderboard.printFormatted();

    leaderboard.add("Sana", 91);
    leaderboard.printFormatted();

    leaderboard.add("Daniyal", 81);
    leaderboard.printFormatted();

    return 0;
}
```

Q8: [5 marks] Write a **recursive** function that takes a positive integer as input and returns the **number of odd digits** in that number.

For example, for input `4923`, the function should return `2` (since `9` and `3` are odd digits).

You must solve the problem using plain recursion:

- No loops
- No global or static variables
- No string conversions

Example:
Input: `4923`
Output: `2`

Function Signature: `int countOddDigits(int n)` (4 marks)

Additional Requirement: (1 mark)
Write a main function that demonstrates your implementation by passing the example input and printing the output.

```
#include <iostream>
using namespace std;

int countOddDigits(int n) {
   if (n == 0) return 0;
   int lastDigit = n % 10;
   int rest = n / 10;
   if (lastDigit % 2 == 1)
       return 1 + countOddDigits(rest);
   else
       return countOddDigits(rest);
}

int main() {
   int number = 4923;
   cout << "Number of odd digits in " << number << " is: " <<
           countOddDigits(number) << endl;
   return 0;
}
```

Q9: [15 marks] Refer to the image below that represents a classic bar phone keypad layout.
Each digit key maps to a group of alphabets (similar to traditional texting).

Write a **recursive backtracking solution** that takes a string containing digits (0–9) and prints all possible letter combinations that the number could represent.

- The digit `'1'` maps to **no letters**
- The digit `'0'` maps to a **space character**

Example:
Input: `"23"`
Output: `"ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"`

Note: The quotation marks in the above example are there just to show that the input/output is a string and they do not need to be in your output

Function Signature: `void getWords(string digits)` (13 marks)

Additional Requirement: (2 marks)
Write a main function that demonstrates your implementation by passing the example input and printing the output.

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void backtrack(const string& digits, int index, const string keypad[],
               string path, vector<string>& result) {
    if (index == digits.size()) {
        result.push_back(path);
        return;
    }

    string letters = keypad[digits[index] - '0'];  // map digit to letters

    for (char letter : letters) {
        backtrack(digits, index + 1, keypad, path + letter, result);
    }
}
```

```
void getWords(const string& digits) {
    const string keypad[10] = {
        " ",     // '0' maps to space
        "",      // '1' has no letters
        "abc",   // '2'
        "def",   // '3'
        "ghi",   // '4'
        "jkl",   // '5'
        "mno",   // '6'
        "pqrs",  // '7'
        "tuv",   // '8'
        "wxyz"   // '9'
    };

    vector<string> result;
    backtrack(digits, 0, keypad, "", result);

    for (const string& word : result) {
        cout << word << " ";
    }
}

int main() {
    string input = "23";
    cout << "Combinations for " << input << ": ";
    getWords(input);
    return 0;
}
```